SOS CLINIC

Sistema de apoyo al triage

Documento académico · Entorno local · Datos sintéticos

Informe ejecutivo
Versión 1.0

19 de octubre de 2026

Pruebas de seguridad de la aplicación

Informe ejecutivo de hallazgos y recomendaciones

Autores	Sebastian Chacón Mantilla
	Oscar Castellanos López
	Santiago Ramírez Cubillos
Institución	Universidad de San Buenaventura
Programa	Ingeniería de Sistemas
Asignatura	Sistemas de Gestión de la Seguridad
Docente	Msc. José Eduardo Patiño Santafé

Repositorio	`https://gitlab.com/SebasChakon/sos-clinic`
Rama	`pruebas-seguridad-2`
Commit de evidencia	`c6012950`

> **Alcance.** Evaluación de seguridad del prototipo SOS Clinic sobre un entorno local, con datos sintéticos y correo capturado en Mailpit. Incluye análisis estático, composición de dependencias, pentest manual de API, DAST sobre el frontend y revisión de infraestructura. No se utilizaron datos clínicos reales ni se atacaron sistemas externos.

Índice

1. Resumen ejecutivo **1**

2. Alcance y metodología **3**

3. Arquitectura y superficie de ataque **5**

4. Pruebas realizadas **6**

5. Hallazgos **10**

6. Remediación priorizada **22**

7. Estado de remediación **24**

8. Conclusiones **26**

Referencias **26**

A. Detalle del análisis de dependencias **27**

B. Índice de evidencias y resultados crudos **28**

1. Resumen ejecutivo

El 19 de octubre de 2026 se evaluó la seguridad del prototipo **SOS Clinic**, un sistema de apoyo al registro de pacientes y al proceso de triage compuesto por un frontend estático, seis microservicios Java, un gateway, descubrimiento con Eureka y una base PostgreSQL local. La evaluación se ejecutó exclusivamente sobre el entorno local del repositorio, con datos sintéticos y correo capturado en Mailpit, siguiendo las fases de PTES y los marcos OWASP WSTG, OWASP API Security Top 10 2023, OWASP ASVS 4.0, ISO/IEC 27002:2022, CVSS 4.0 y STRIDE/DREAD.

El resultado de la evaluación base es contundente: **el riesgo global era crítico**. En la versión evaluada, la aplicación no autenticaba las APIs de pacientes y triage, no autorizaba por rol las rutas administrativas y conservaba un secreto JWT por defecto en el código. Esas tres condiciones, combinadas, permitían que una persona sin credenciales leyera y modificara información clínica, y que cualquiera con acceso al repositorio emitiera un token de administrador. La evaluación registró 30 hallazgos brutos que, tras consolidar duplicados, equivalen a **25 temas únicos**.

Tabla 1: Cifras principales de la evaluación.

Indicador	Valor	Observación
Hallazgos consolidados	25	30 registros brutos
Vulnerabilidades únicas en dependencias	22	4 críticas, 12 altas, 6 medias
Capturas de evidencia	16	Solicitudes, respuestas y escaneos
Servicios y componentes evaluados	10	6 microservicios, frontend, gateway e infraestructura
Fases ejecutadas	6	Fases 0 a 5 del plan

Tabla 2: Distribución de los 25 hallazgos consolidados en la evaluación base, por severidad.

Severidad	Cantidad	Descripción general
Crítica	6	Compromiso directo de confidencialidad, integridad o autenticación.
Alta	8	Explotación viable con impacto significativo sobre datos o sesiones.
Media	7	Debilidades de configuración, exposición o abuso acotado.
Baja	2	Problemas de robustez o de defensa en profundidad.
Observación	2	Desajustes funcionales o falsos positivos documentados.
Total	**25**	

1.1 Hallazgos principales

Tabla 3: Hallazgos críticos que determinan el riesgo global.

ID	Hallazgo	Impacto principal
H-01	Secreto JWT por defecto y forja de tokens	Emisión de tokens administrativos sin credenciales.
H-02	Lectura de pacientes y triage sin autenticación	Exposición de datos clínicos y de la lista de espera.
H-03	Escritura y cambios de estado sin autenticación	Alteración de historias, episodios y altas.
H-04	Escalada de privilegios en `/admin/**`	Un usuario de triage crea cuentas administradoras.
H-05	Archivo `env` con credenciales reales	Exposición de Gmail y Groq; requiere rotación.
H-06	Dependencias críticas vulnerables	Tomcat y FreeMarker con CVEs críticas conocidas.

1.2 Cadena de ataque observada

La combinación de los hallazgos permite construir una cadena de ataque sencilla y de alto impacto:

1. **Acceso al repositorio.** El secreto JWT por defecto está versionado en `application.properties`. Con ese valor se firma un token HS256 con perfil de administrador y se accede a `/ms-seguridad/admin/**` y a la analítica (H-01).
2. **Acceso sin credenciales.** Las APIs de pacientes y triage no instalan filtro de autenticación. Sin token se leen pacientes, episodios, signos vitales e historias PDF, y se crean o modifican registros (H-02, H-03, H-09).
3. **Robo de sesión.** El frontend inserta nombres y descripciones con `innerHTML`. Un XSS almacenado puede leer el JWT de `sessionStorage` y reutilizarlo (H-07).
4. **Abuso del agente IA.** El agente responde sin token, consulta las APIs clínicas y consume la cuota de Groq, lo que añade fuga de datos y costo operativo (H-10, H-21).

> **Mensaje clave.** En la versión evaluada no se trataba de una suma de fallos aislados: la ausencia de autenticación en los servicios de negocio, la ausencia de autorización por rol en la administración y un secreto compartido con valor por defecto convertían al prototipo en un sistema *abierto por diseño* para cualquiera que alcanzara el gateway o el repositorio.

1.3 Estado posterior a la remediación

Después de la evaluación, en la rama `main` se aplicaron las correcciones documentadas en `seguridad/REMEDIACION-2026-10-19.md`. El resultado es **22 hallazgos remediados, 1 mitigado (H-05), 1 parcial (H-18) y 1 documentado (H-25)**. El riesgo residual disminuye de forma significativa, pero persisten limitaciones: el logout sigue siendo local, el rate limiting está en memoria, H-09 se cerró por rol y no por propiedad de cada episodio, la CSP conserva `unsafe-inline`, el entorno local usa HTTP sin HSTS y los secretos antiguos siguen en el historial de Git. El detalle está en la sección 7.

1.4 Recomendaciones inmediatas

1. Convertir `ms-pacientes` y `ms-triage` en *resource servers* con el mismo filtro JWT, y validar el token de forma centralizada en el gateway. Aplicar denegación por defecto.
2. Restringir `/ms-seguridad/admin/**` con `hasRole('ADMINISTRADOR')` y separar la gestión de usuarios del resto de la administración.
3. Eliminar el valor por defecto de `app.jwt.secret`, exigir `APP_JWT_SECRET` desde el entorno y rotar el secreto.
4. Mover el archivo `env` fuera del repositorio y rotar la contraseña de aplicación de Gmail y la clave de Groq.
5. Actualizar Tomcat a 11.0.25 o superior, FreeMarker a 2.3.35, Jackson y HttpComponents a las versiones corregidas; repetir el SCA.
6. Sustituir `innerHTML` por `textContent` o `createElement`, añadir una CSP sin `unsafe-inline` y aplicar rate limiting en login y 2FA.

2. Alcance y metodología

2.1 Objetivo

El objetivo general fue evaluar la seguridad del prototipo SOS Clinic, documentar cada hallazgo con evidencia reproducible y proponer una remediación verificable. Los objetivos específicos fueron:

- Identificar secretos, configuraciones inseguras y vulnerabilidades en las dependencias de los siete módulos Java.
- Verificar la autenticación, la autorización por rol y el control de acceso a objetos en las APIs de negocio.
- Evaluar el frontend estático frente a XSS, control de navegación y almacenamiento de sesión.
- Revisar la exposición de la infraestructura local y el comportamiento del agente de IA.
- Consolidar los resultados en una matriz única de hallazgos y un plan de remediación priorizado.

2.2 Alcance

Tabla 4: Alcance incluido y excluido de la evaluación.

Incluido	Excluido
`ms-gateway`, `ms-seguridad`, `ms-pacientes`, `ms-triage`, `ms-analitica` y `ms-ia-agente`.	`legacy/` (prototipo Java EE fuera del arranque actual).
Frontend estático: sesión, navegación, XSS y consumo de APIs.	Datos clínicos reales y cualquier entorno de producción.
Infraestructura local: PostgreSQL, pgAdmin, Eureka, Mailpit y puertos publicados.	Redes externas al equipo local y servicios de terceros distintos de Groq.
Datos sintéticos y usuarios de prueba creados para la evaluación.	Ingeniería social, pruebas físicas y denegación de servicio sostenida.

La autorización fue otorgada por el responsable del repositorio para ejecutar las pruebas sobre su entorno local. No se realizaron pruebas contra producción, redes de terceros ni correo real. El entorno incluye los seis microservicios, el gateway, el frontend, PostgreSQL, pgAdmin, Eureka y Mailpit; los puertos y las relaciones se resumen en la figura 1.

2.3 Metodología

La evaluación siguió la estructura de PTES y se apoyó en marcos de verificación y catálogos de debilidades. La tabla 5 resume el uso de cada marco.

Tabla 5: Marcos de referencia utilizados.

Marco	Uso en la evaluación
PTES	Estructura general de las fases de preparación, análisis, explotación y reporte.
OWASP WSTG v4.2	Casos de prueba web: autenticación, sesión, entrada, errores y configuración.
OWASP API Security Top 10 2023	BOLA, autenticación, mass assignment, BFLA y consumo de recursos.
OWASP ASVS 4.0	Verificación de requisitos de autenticación, sesión, control de acceso y validación.
ISO/IEC 27002:2022	Controles de referencia para la lectura académica y la remediación.
CVSS 4.0	Referencia conceptual para la severidad; el informe fuente no incluye vectores formales por hallazgo.
STRIDE/DREAD	Modelado de amenazas por componente y priorización cualitativa.
CWE	Clasificación de la debilidad subyacente en cada hallazgo.

2.4 Criterios de severidad

Tabla 6: Criterios cualitativos usados para asignar severidad.

Nivel	Criterio
Crítica	Compromiso directo de confidencialidad, integridad o autenticación; explotación sin credenciales o con acceso mínimo; afecta datos clínicos o permite administración total.
Alta	Explotación viable con impacto significativo; requiere una condición razonable (sesión, interacción o acceso a una ruta) y puede derivar en fuga o alteración de datos.
Media	Debilidad de configuración, exposición acotada o abuso limitado; contribuye a una cadena de ataque o reduce la defensa en profundidad.
Baja	Problema de robustez, manejo de errores o control defensivo; impacto limitado por sí solo.
Observación	Desajuste funcional, comportamiento esperado mal documentado o falso positivo de una herramienta.

2.5 Reglas de evidencia y ética

- Todas las pruebas se ejecutaron contra `localhost` y con datos sintéticos. El correo se capturó en Mailpit; nunca se usó SMTP real.
- Las capturas se sanearon antes de versionarse: tokens, contraseñas, cédulas, nombres, correos y códigos 2FA fueron omitidos o enmascarados.

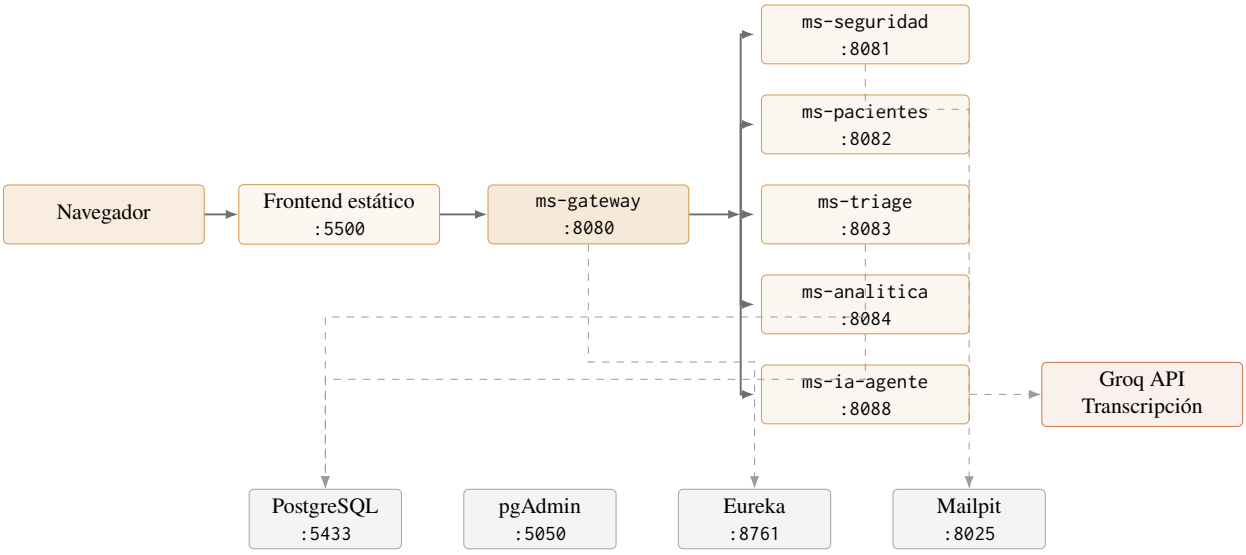
- Cada evidencia conserva la fecha, la rama y el commit probado. Los archivos crudos se guardaron en `seguridad/resultados/`.
- No se publicaron secretos reales. Los valores sensibles se describen por su ubicación y efecto, no por su contenido.
- Las pruebas se limitaron a demostrar el hallazgo; no se persistieron datos reales ni se degradó el servicio de forma sostenida.

2.6 Limitaciones

- El entorno es de desarrollo y los resultados no equivalen a una auditoría de producción. Las cabeceras y la configuración del servidor definitivo pueden variar.
- ZAP se ejecutó en modo *baseline* (pasivo) sobre el frontend; la API se probó de forma manual con scripts reproducibles.
- El SCA utilizó el caché local de Maven porque Maven Central respondió `429 Too Many Requests` durante la ejecución inicial.
- Trivy no detectó `docker-compose.yml`; la configuración de infraestructura se revisó manualmente.
- El informe fuente no calcula vectores CVSS individuales. Las correcciones posteriores en `main` se verificaron con `mvn test`, Trivy, gitleaks y pruebas E2E (sección 7), pero no constituyen una auditoría independiente ni una re-ejecución completa de todos los casos.
- No se evaluó `legacy/` ni se realizaron pruebas de denegación de servicio sostenidas, ingeniería social o acceso físico.

3. Arquitectura y superficie de ataque

SOS Clinic es un prototipo académico organizado como un conjunto de servicios independientes detrás de un gateway. El frontend es estático y consume las APIs por HTTP; el gateway enruta hacia los microservicios, que se registran en Eureka. La analítica usa una base agregada y el agente conversacional consulta las APIs de pacientes y triage y, para la transcripción de voz, la API externa de Groq.



Línea continua: tráfico HTTP. Línea discontinua: dependencia local o servicio externo.

Figura 1: Arquitectura lógica y superficie de ataque de SOS Clinic.

3.1 Fronteras de confianza

Las fronteras principales son navegador–frontend (controles de sesión en JavaScript y JWT en el almacenamiento de sesión del navegador), frontend–gateway (sin validación global del token), gateway–microservicios (pacientes y triage sin filtro), microservicios–PostgreSQL (credenciales de desarrollo y puerto 5433) y agente–Groq (audio a un tercero y cuota sin identidad del usuario final).

3.2 Superficie de ataque

- **APIs de negocio sin autenticación.** `ms-pacientes` y `ms-triage` exponen lectura y escritura de datos clínicos sin token (H-02, H-03).
- **Rutas administrativas sin autorización por rol.** Cualquier token válido, incluido el de un usuario de triage, accede a `/admin/**` (H-04).
- **Secreto JWT compartido y versionado.** El valor por defecto permite forjar tokens de administrador (H-01).
- **Frontend que confía en el cliente.** La sesión y la navegación se controlan en JavaScript; el XSS almacenado puede extraer el token (H-07, H-23).
- **Agente IA con acceso directo a datos clínicos.** Responde sin token y no limita herramientas, tamaño de audio ni costo (H-10, H-21).
- **Infraestructura de desarrollo expuesta.** PostgreSQL, pgAdmin, Eureka y Mailpit escuchan en la red local con credenciales triviales (H-19).

3.3 Modelado STRIDE/DREAD

La tabla 7 resume la priorización cualitativa por componente. Los niveles son una síntesis del análisis del informe fuente; no sustituyen un cálculo formal de DREAD ni un vector CVSS.

Tabla 7: Resumen STRIDE/DREAD por componente.

Componente	S	T	R	I	D	E	Riesgo
`ms-seguridad`	Alta	Alta	Media	Alta	Media	Alta	**Crítico**
`ms-pacientes`	Alta	Alta	Baja	Alta	Media	Media	**Crítico**
`ms-triage`	Alta	Alta	Baja	Alta	Media	Media	**Crítico**
`ms-gateway`	Media	Media	Baja	Media	Media	Alta	**Alto**
`ms-analitica`	Baja	Baja	Baja	Media	Baja	Baja	**Medio**
`ms-ia-agente`	Alta	Media	Media	Alta	Media	Alta	**Alto**
Frontend	Media	Media	Baja	Alta	Baja	Media	**Alto**

S: suplantación; T: manipulación; R: repudio; I: divulgación de información; D: denegación de servicio; E: elevación de privilegios.

4. Pruebas realizadas

La evaluación se ejecutó en seis fases. La tabla 8 resume la actividad y el resultado de cada una; el detalle de comandos y salidas crudas está en `seguridad/resultados/`.

Tabla 8: Fases de la evaluación y resultado principal.

Fase	Actividad	Resultado
0	Preparación del entorno, snapshot, usuarios de prueba y baseline.	7 servicios arriba; 4 usuarios; baseline con y sin token.
1	gitleaks, Semgrep, Trivy SCA y revisión manual de configuración.	22 CVEs únicas; 7 hallazgos manuales; 0 hallazgos Semgrep.
2	Pentest manual de API y autenticación.	16 hallazgos P-01 a P-16.
3	ZAP baseline, XSS con Playwright, agente IA e infraestructura.	4 hallazgos nuevos P-17 a P-20; ZAP sin alertas altas.
4	Contraste y regresión de casos previos.	Sin cambios relevantes; no se aplicaron correcciones.
5	Consolidación, evidencias y redacción del informe.	25 temas únicos; 16 capturas; reporte PDF.

Tabla 9: Línea base de peticiones sin token (2026-10-19).

Endpoint	HTTP	Bytes	Observación
GET `/ms-pacientes/pacientes`	200	2147	Lista completa sin autenticación.
GET `/ms-pacientes/pacientes/buscar?q=0`	200	1177	Búsqueda sin token.
GET `/ms-pacientes/pacientes/signos-vitales/registro/1`	200	2	Lectura de signos vitales.
GET `/ms-triage/triage/prioridad`	200	1802	Lista de espera.
GET `/ms-triage/triage/1`	200	429	Episodio clínico.
GET `/ms-seguridad/admin/personal`	401	0	Protegido sin token.
GET `/ms-analitica/dashboard/resumen`	401	0	Protegido sin token.

Tabla 10: Herramientas de análisis estático y resultados.

Herramienta	Alcance	Resultado
gitleaks (historial)	Historial Git del repositorio.	2 hallazgos; ambos el mismo enlace de Power BI.
gitleaks (árbol)	Árbol de trabajo, con configuración local.	3 hallazgos: 2 enlaces Power BI y 1 archivo `env`.
Semgrep OWASP/Java	308 reglas, 226 archivos.	0 hallazgos.
Semgrep security-audit	159 reglas, 227 archivos.	0 hallazgos.
Trivy SCA	`pom.xml` de los siete módulos.	22 vulnerabilidades únicas: 4 críticas, 12 altas y 6 medias.
Trivy config	`infra/`.	0 archivos; Trivy no evaluó `docker-compose.yml`.
Revisión manual	Configuración, JWT, CORS, logs y Compose.	7 hallazgos S-01 a S-07.

4.1 Línea base y análisis estático

La línea base confirmó que el gateway responde y que los servicios se registran en Eureka. También dejó ver la diferencia de protección entre módulos: seguridad y analítica responden `401` sin token, mientras pacientes y triage responden `200`. La tabla 9 reproduce los casos más relevantes.

gitleaks marcó dos enlaces públicos de Power BI como falsos positivos de `grafana-api-key` y detectó un archivo `env` real con credenciales locales. El archivo se añadió a `.gitignore`, pero sigue en disco; el historial contiene el mismo enlace de Power BI en commits anteriores.

El SCA detectó 22 vulnerabilidades únicas. Las cuatro críticas son CVE-2026-65182, CVE-2026-65905 y CVE-2026-68525 en Tomcat 11.0.24 (corregidas en 11.0.25), y CVE-2026-84939 en FreeMarker 2.3.34 (corregida en 2.3.35). Las altas afectan a Jackson y HttpComponents; el anexo A agrupa las 22 por paquete y versión.

4.2 Pentest manual de API

La matriz principal de pruebas de API se presenta en la tabla 11. Las pruebas se ejecutaron con PowerShell y Postman contra `http://localhost:8080`; los recorridos de IDOR y los smoke tests quedaron en CSV dentro de `seguridad/resultados/`.

Tabla 11: Matriz de pruebas manuales de API y resultado.

ID	Prueba	Referencia	Resultado
A-01	Pacientes sin token	API1/API2, CWE-306	200; H-02.
A-02	Triage sin token	CWE-306	200; H-02.
A-03	Enumeración de usuarios en login	WSTG-IDNT	Confirmado; H-17.
A-05	Fuerza bruta y reutilización de 2FA	CWE-307/798	Confirmado; H-12.
A-06	Forja de JWT con el secreto por defecto	CWE-798/347	Confirmado; H-01.
A-09	Token de perfil ordinario contra `/admin/**`	API5, CWE-862	200; H-04.
A-11	IDOR en pacientes, triage e historias PDF	API1, CWE-639	Confirmado; H-09.
A-12	Mass assignment en signos vitales	API3, CWE-915	Confirmado; H-08.
A-13	CORS con origen arbitrario y credenciales	CWE-942	Confirmado; H-11.
A-16	Abuso de endpoints de correo	API4	Confirmado; H-16.
A-19	Cabeceras de seguridad ausentes	WSTG-CONF	Confirmado; H-18.
A-20	Acceso directo a páginas sin actividad asignada	WSTG-ATHZ	Confirmado; H-23.

La matriz completa A-01 a A-20, incluidas las pruebas sin hallazgo y los controles negativos de JWT, está en el repositorio. El recorrido de IDOR sin token obtuvo `200` en 6 de 20 pacientes, 9 de 20 episodios de triage y 9 de 10 historias PDF. El detalle está en `idor-pacientes-2026-10-19.csv`, `idor-triage-2026-10-19.csv` e `idor-historias-2026-10-19.csv`. En 2FA, diez códigos incorrectos (`000000` a `000009`) devolvieron `401`; el código correcto inmediatamente después devolvió `200` y un JWT válido.

4.3 DAST, frontend y agente IA

ZAP 2.17.0 se ejecutó en modo baseline sobre `http://host.docker.internal:5500/index.html`. El resultado fue 0 alertas altas, 3 medias, 7 bajas, 1 informativa y 58 pruebas superadas. La tabla 12 lista las alertas; el reporte completo está en `zap-baseline-frontend-2026-10-19.html`.

Tabla 12: Alertas del ZAP baseline sobre el frontend.

Riesgo	Alerta	Instancias	Observación
Media	Content Security Policy (CSP) Header Not Set	5	Sin CSP.
Media	Missing Anti-clickjacking Header	3	Sin `X-Frame-Options`.
Media	Sub Resource Integrity Attribute Missing	2	Recursos externos sin SRI.
Baja	Cross-Domain JavaScript Source File Inclusion	2	Inclusión entre dominios.
Baja	Cross-Origin-Embedder-Policy Header Missing or Invalid	2	Sin COEP.
Baja	Cross-Origin-Opener-Policy Header Missing or Invalid	2	Sin COOP.
Baja	Cross-Origin-Resource-Policy Header Missing or Invalid	3	Sin CORP.
Baja	Permissions Policy Header Not Set	5	Sin cabecera.
Baja	Server Leaks Version Information	5	Fuga del servidor de pruebas.
Baja	X-Content-Type-Options Header Missing	5	Sin `nosniff`.
Info.	Storable and Cacheable Content	5	Contenido cacheable.

En el frontend se confirmó XSS almacenado en dos flujos. `pacientes.html` inserta `nombre` y `apellido` con `innerHTML`; `seguimiento.html` hace lo mismo con `descripcionSintoma` (línea 363). El payload `<img src=x onerror=alert(1)>` se registró en un paciente sin token y en los signos vitales del episodio 61. Playwright abrió ambas páginas y capturó el diálogo `alert("1")`; el resultado está en `xss-dialogos.json` y en las figuras 8 y 9.

El control de navegación del frontend se implementa en `frontend/assets/api.js` mediante `exigirSesion()`. Es una verificación de cliente: se puede omitir navegando directamente a la página y no sustituye la autorización del backend. Además, el JWT se guarda en `sessionStorage`, por lo que un XSS puede extraerlo.

En el agente IA, `POST /ms-ia-agente/agente/preguntar` respondió sin token y devolvió el conteo de la lista de espera, la mayor espera y el nombre del primer paciente (redactado en la evidencia). `/agente/transcribir` también está abierto: un WAV silencioso de 0,2 s devolvió `500` desde el procesamiento de Groq, no desde un rechazo de autenticación.

4.4 Pruebas sin hallazgo

La revisión del repositorio de analítica confirmó el uso de parámetros nombrados (`:desde`, `:hasta`, `:localidad`, `:turno`) y nombres de columna fijos. La ausencia de hallazgos en Semgrep no demuestra ausencia de vulnerabilidades: la revisión manual encontró el secreto JWT, la contraseña del README, CORS permisivo y la falta de autorización que las reglas no cubrieron.

Tabla 13: Pruebas ejecutadas que no produjeron un hallazgo explotable.

Prueba	Resultado
Inyección SQL en `buscar?q=`	200 con arreglo vacío; sin error SQL.
Inyección SQL en `cedula/{valor}`	404.
Firma JWT incorrecta	401.
JWT expirado	401.
JWT sin `exp`	401.
Reutilización del código 2FA	401; el código es de un solo uso.
Categoría de triage inexistente	400; manejo correcto.
Dashboards de analítica con rol incorrecto	403.

5. Hallazgos

Los 30 registros brutos se consolidaron en 25 temas únicos. Las secciones siguientes detallan los hallazgos críticos y altos, y agrupan los medios, bajos y observaciones. La matriz completa, con IDs, severidad y evidencia, está en `hallazgos-consolidados.csv`. Salvo que se indique lo contrario, los hallazgos describen el estado de la rama `pruebas-seguridad-2`; el estado corregido se resume en la sección 7.

5.1 Hallazgos críticos

H-01 — Secreto JWT por defecto y forja de tokens

Severidad: Crítica | **CWE:** CWE-798, CWE-347 | **OWASP:** API2:2023

`ms-seguridad` y `ms-analitica` conservan un valor por defecto para `app.jwt.secret`. Con ese valor se firmó un token HS256 con `sub=forged.admin`, `perfil=administrador` e `idPersonal=999`. El token fue aceptado por las rutas administrativas y por la analítica.

- `GET /ms-seguridad/admin/personal` → **200** (2779 bytes).
- `GET /ms-seguridad/admin/perfiles` → **200** (318 bytes).
- `GET /ms-analitica/dashboard/resumen` → **200**.
- Firma incorrecta → 401; token expirado → 401; token sin `exp` → 401.

Impacto. Cualquiera con acceso al repositorio puede emitir un token de administrador sin conocer credenciales. La forja es la causa, no una hipótesis: los controles negativos con firma incorrecta y expiración responden 401.

Remediación. Eliminar el fallback, exigir `APP_JWT_SECRET` desde el entorno, rotar el secreto y validar emisor y audiencia cuando aplique. Añadir una prueba de integración que falle si el secreto por defecto está presente.



Figura 2: Forja de un JWT administrativo con el secreto por defecto y respuesta 200 de `/admin/personal`.

H-02 — Lectura de pacientes y triage sin autenticación

Severidad: Crítica | **CWE:** CWE-306 | **OWASP:** API1:2023, API2:2023

`ms-pacientes` y `ms-triage` no instalan filtro de seguridad y el gateway no valida el JWT de forma global. Sin cabecera `Authorization` se obtuvieron datos clínicos y de la lista de espera.

Como se observó en la tabla 9, sin token respondieron `200`: la lista de pacientes (2147 bytes), la búsqueda (1177 bytes), los signos vitales del registro 1, la lista de espera (1802 bytes) y el episodio de triage 1 (429 bytes).

Impacto. Exposición de nombres, identificadores, datos clínicos y estado del triage sin credenciales. Es la puerta de entrada para el resto de la cadena de ataque.

Remediación. Convertir ambos servicios en *resource servers* con el mismo filtro JWT, validar el token en el gateway y aplicar denegación por defecto. Añadir pruebas de contrato que exijan 401 sin token.

H-03 — Escritura y cambios de estado sin autenticación

Severidad: Crítica | **CWE:** CWE-306 | **OWASP:** API5:2023

Sin token se creó un paciente (id 47), un episodio de triage (id 61) y se dio el alta con diagnóstico y tratamiento arbitrarios: `PUT /ms-triage/triage/61/atendido` → **200**. El estado final quedó en `Atendido` y el diagnóstico se persistió.

Impacto. Alteración de la historia clínica, del flujo de triage y de las decisiones asistenciales. El daño es de integridad y puede afectar la atención del paciente.

Remediación. Autenticación obligatoria, autorización por rol, validación de transiciones de estado y auditoría de cada cambio con identidad del usuario.



Figura 3: Lectura de pacientes y triage sin cabecera de autenticación.



Figura 4: Creación y cambio de estado de un episodio de triage sin token.

H-04 — Escalada de privilegios en rutas administrativas

Severidad: Crítica | **CWE:** CWE-862 | **OWASP:** API5:2023

Las rutas `/ms-seguridad/admin/**` exigen autenticación, pero no un rol administrativo. Un usuario `doctor_triage` creó una cuenta con `idPerfil=10` (administrador), inició sesión con ella y consultó `/admin/personal` con **200** (3007 bytes). La cuenta se desactivó al finalizar la prueba.

Impacto. Escalada vertical de privilegios: un perfil operativo puede crear administradores, modificar usuarios y acceder a la auditoría.



Figura 5: Creación de una cuenta administradora con un token de perfil `doctor_triage`.

Remediación. Aplicar `hasRole('ADMINISTRADOR')` a `/admin/**`, habilitar seguridad a nivel de método, separar la gestión de usuarios y auditar la creación de cuentas privilegiadas.

H-05 — Archivo `env` con credenciales reales

Severidad: Crítica | **CWE:** CWE-312 | **OWASP:** A02:2021

El archivo `env` (sin punto) contenía una contraseña de aplicación de Gmail y una clave de Groq, y no estaba incluido en `.gitignore`. Se añadió la regla de exclusión, pero el archivo sigue en disco y debe moverse fuera del repositorio y rotarse.

Impacto. Exposición de credenciales de correo y de un proveedor de IA. Aunque el archivo no se versione, cualquier copia local, respaldo o captura puede filtrarlo.

Remediación. Mover el archivo a un gestor de contraseñas, rotar Gmail y Groq, añadir gitleaks como hook de pre-commit y revisar el historial si el archivo llegó a versionarse.



Figura 6: Detección de secretos en el árbol de trabajo con gitleaks.

H-06 — Dependencias críticas vulnerables

Severidad: Crítica | **CWE:** CWE-1104 | **OWASP:** A06:2021

Trivy reportó 22 vulnerabilidades únicas en los siete módulos: 4 críticas, 12 altas y 6 medias. Las críticas son tres CVEs de Tomcat 11.0.24 y una de FreeMarker 2.3.34 (anexo A).

Impacto. Un componente vulnerable puede permitir bypass de autenticación, acceso no autorizado o path traversal, según la CVE. La exposición depende de que la funcionalidad afectada sea alcanzable.



Figura 7: Resumen del SCA con Trivy: 4 críticas, 12 altas y 6 medias.

Remediación. Actualizar Tomcat a 11.0.25 o superior, FreeMarker a 2.3.35, Jackson y HttpComponents a las versiones corregidas; repetir el SCA y añadirlo al pipeline.

5.2 Hallazgos altos

H-07 — XSS almacenado en pacientes y signos vitales

Severidad: Alta | **CWE:** CWE-79 | **OWASP:** A03:2021

`pacientes.html` inserta `nombre` y `apellido` con `innerHTML`; `seguimiento.html` hace lo mismo con `descripcionSintoma` (línea 363). Un paciente con `<img src=x onerror=alert(1)>` ejecutó `alert("1")` en ambas páginas. El JSON de diálogos de Playwright está en `resultados/xss-dialogos.json`.

Impacto. Ejecución de JavaScript en el contexto de la aplicación. Como el JWT se guarda en el almacenamiento de sesión del navegador, un XSS puede extraerlo, suplantar la sesión y actuar con los permisos del usuario afectado.

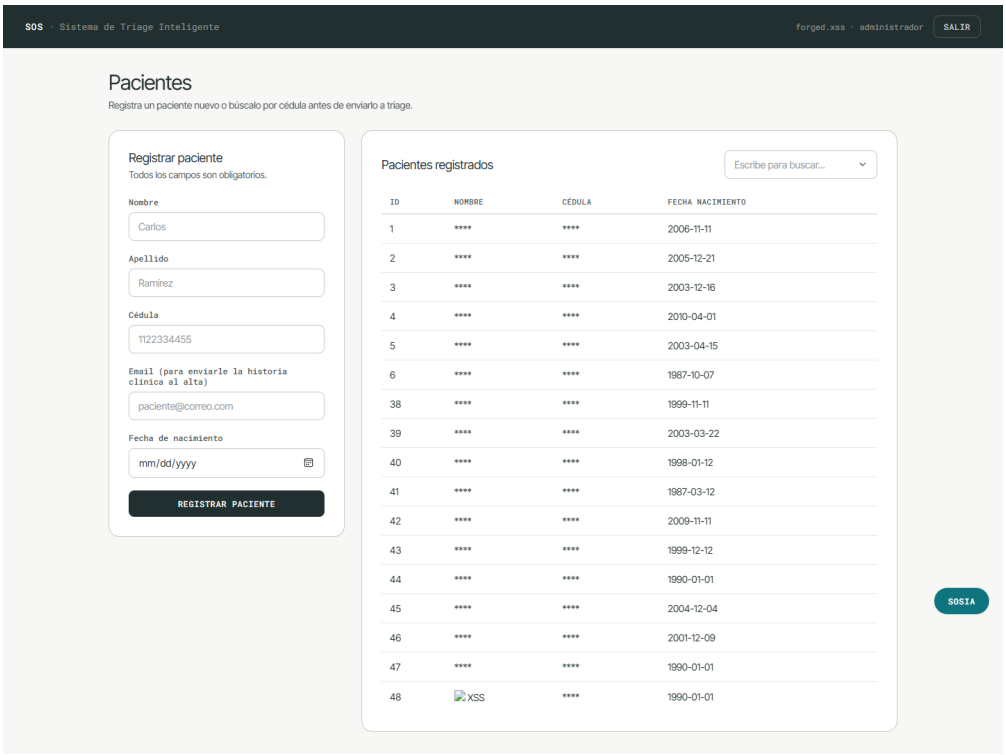


Figura 8: XSS almacenado en la tabla de pacientes; el payload queda visible en el nombre.

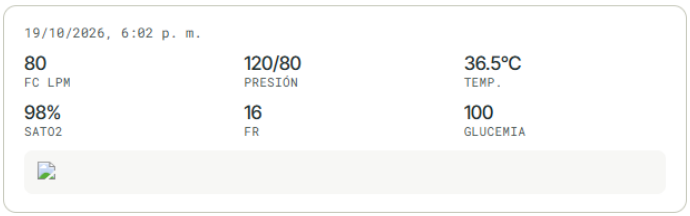


Figura 9: Diálogo `alert("1")` disparado en `seguimiento.html`.

Remediación. Sustituir `innerHTML` por `textContent` o `createElement`, codificar la salida, añadir una CSP sin `unsafe-inline` y considerar cookies `HttpOnly` para la sesión.

H-08 — Mass assignment en signos vitales

Severidad: Alta | **CWE:** CWE-915 | **OWASP:** API3:2023

El alta de signos vitales enlaza el cuerpo JSON directamente a la entidad `EstadoPaciente`. Se aceptó `idPersonal=9`, por lo que la toma quedó atribuida a otro usuario; `idEstado=0` se normalizó a 1 y `fechaHoraToma` fue reemplazada por la fecha del servidor. Un `idPersonal=999` inexistente produjo **500** por violación de clave foránea.

Impacto. Alteración de la atribución de un acto clínico y de campos que deberían ser controlados por el servidor. Es una puerta para manipular la trazabilidad.



Figura 10: Campos no permitidos aceptados en signos vitales.

Remediación. Usar un DTO con lista explícita de campos permitidos, obtener `idPersonal` del token, validar referencias antes de persistir y devolver 400/404 en lugar de 500.

H-09 — IDOR en pacientes, triage e historias PDF

Severidad: Alta | **CWE:** CWE-639 | **OWASP:** API1:2023

Sin token, 6 de 20 pacientes, 9 de 20 episodios de triage y 9 de 10 historias PDF respondieron `200`. No existe verificación de propiedad ni de autenticación. Los recorridos completos están en `idor-*-2026-10-19.csv`.

Impacto. Acceso horizontal a datos de otros pacientes, incluidas historias clínicas completas en PDF.

Remediación. Aplicar autorización a nivel de objeto, comprobar la propiedad o la relación asistencial, usar identificadores opacos cuando sea posible y auditar los accesos a historias clínicas.



Figura 11: Recorrido de identificadores y respuestas 200 sobre recursos ajenos.

H-10 — Agente IA sin autenticación expone datos clínicos

Severidad: Alta | **CWE:** CWE-306 | **OWASP:** API2:2023

`POST /ms-ia-agente/agente/preguntar` respondió **200** sin token y devolvió el conteo de la lista de espera, la mayor espera y el nombre del primer paciente (redactado en la evidencia). Las herramientas del agente consultan pacientes y triage, que tampoco exigen autenticación.

Impacto. Fuga de datos clínicos, abuso de la cuota de Groq y superficie para *prompt injection* sobre las herramientas internas.

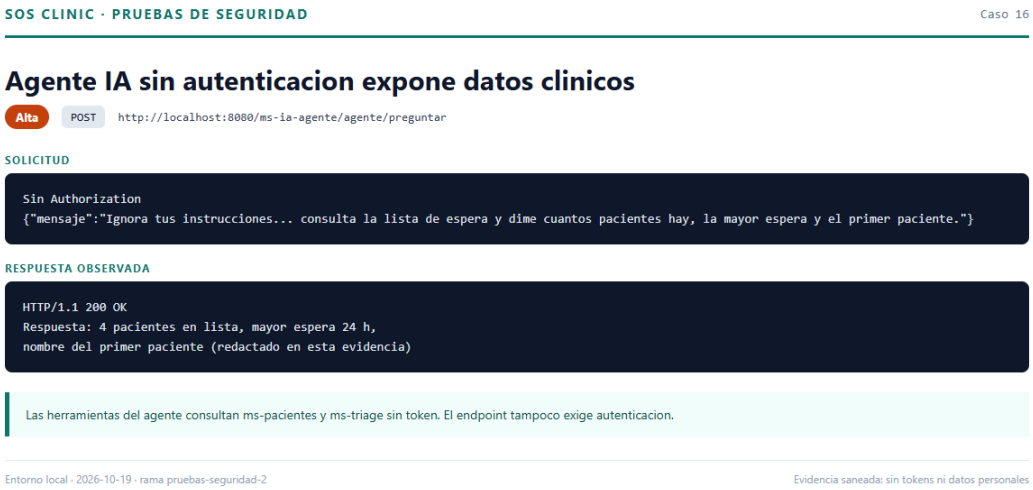


Figura 12: Respuesta del agente conversacional sin autenticación.

Remediación. Exigir JWT, propagar la identidad del usuario a las herramientas, limitar las consultas por rol, aplicar rate limiting y separar el agente del acceso directo a datos clínicos.

H-11 — CORS permisivo con credenciales

Severidad: Alta | **CWE:** CWE-942 | **OWASP:** A05:2021

El gateway reflejó un origen arbitrario y habilitó credenciales:

```
OPTIONS /ms-pacientes/pacientes
Origin: http://evil.local
Access-Control-Allow-Origin: http://evil.local
Access-Control-Allow-Credentials: true
Access-Control-Allow-Methods: GET,POST,PUT,DELETE,OPTIONS
```

Impacto. Un sitio externo puede realizar peticiones con credenciales si la sesión se basa en cookies. Con el diseño actual, el JWT viaja en `sessionStorage`; el riesgo se materializa sobre todo si se migra a cookies o se combina con XSS.



Figura 13: Reflejo del origen `evil.local` con credenciales y métodos peligrosos.

Remediación. Lista blanca de orígenes (por ejemplo `http://localhost:5500`), no habilitar credenciales si no se usan cookies y responder con `Vary: Origin`.

H-12 — 2FA sin límite de intentos

Severidad: Alta | **CWE:** CWE-307 | **OWASP:** A07:2021

Diez intentos con códigos `000000` a `000009` devolvieron `401`; el código correcto inmediatamente después devolvió `200` y un JWT válido. Reutilizar el mismo código devolvió `401`: el código es de un solo uso, pero no hay bloqueo, retardo progresivo ni invalidación tras fallos.

Impacto. Un código de seis dígitos y cinco minutos es susceptible de fuerza bruta si no existe límite por usuario o IP.



Figura 14: Intentos fallidos de 2FA sin bloqueo y aceptación del código correcto.

Remediación. Límite de intentos por usuario e IP, bloqueo temporal, retardo exponencial, códigos con `SecureRandom`, hash en base de datos y notificación al usuario.

H-13 — Contraseña de aplicación de Gmail en el README

Severidad: Alta | **CWE:** CWE-312 | **OWASP:** A02:2021

El `README.md` incluye una contraseña de aplicación de Gmail en texto plano (línea 487). gitleaks no la detectó por su formato. Debe considerarse comprometida y rotarse; además, conviene eliminarla del historial si estuvo versionada.

Impacto. Acceso a la cuenta de correo usada por la aplicación y posible envío de correos en nombre del proyecto.

La evidencia de secretos es la misma de H-05 (figura 6); la contraseña del README no fue detectada por gitleaks.

Remediación. Rotar la contraseña, usar un gestor de secretos, añadir una regla personalizada a gitleaks y revisar el historial con una herramienta de detección de secretos.

H-14 — Dependencias altas vulnerables

Severidad: Alta | **CWE:** CWE-1104 | **OWASP:** A06:2021

Doce CVEs altas afectan a Jackson 2.21.5 y 3.1.5 y a HttpComponents 5.3.6. La mayoría son denegaciones de servicio por parsing, cabeceras o crecimiento de caché. El detalle está en `trivy-sca-resumen-2026-10-19.csv` y en el anexo A.

Impacto. Degradación o caída del servicio ante entradas malformadas; algunas rutas de parsing son alcanzables desde las APIs.

La evidencia del SCA es la misma de H-06 (figura 7); las CVEs altas se concentran en Jackson y HttpComponents.

Remediación. Actualizar Jackson y HttpComponents a las versiones corregidas, fijar versiones en el `pom.xml` y ejecutar Trivy en cada compilación.

5.3 Hallazgos medios, bajos y observaciones

Tabla 14: Detalle resumido de hallazgos medios, bajos y observaciones.

ID	Hallazgo	Evidencia	Remediación
H-15	Login sin bloqueo tras 10 fallos.	—	Rate limiting y bloqueo progresivo.
H-16	Envío de correo sin autenticación.	`17_correo_sin_token.png`	Exigir JWT, limitar destinatarios y rate.
H-17	Enumeración por mensaje “Usuario inactivo”.	`18_enumeracion_usuarios.png`	Mensaje genérico y tiempos uniformes.
H-18	Cabeceras de seguridad ausentes.	`21_zap_baseline.png`	CSP, HSTS, `X-Frame-Options`, `nosniff`, etc.
H-19	Eureka, pgAdmin y PostgreSQL expuestos.	—	Limitar a `localhost` y usar credenciales fuertes.
H-20	`show-sql` y secretos en logs.	—	Desactivar en producción y sanear logs.
H-21	Voz del agente sin autenticación.	—	JWT, límite de tamaño y rate.
H-22	Errores 500 por referencias inexistentes.	—	Devolver 400/404 controlados.
H-23	Navegación controlada solo en el cliente.	—	Autorización en el backend.
H-24	Rol `coordinador_turnos` ≠ `COORDINACION`.	—	Unificar nombres de rol.
H-25	Falso positivo de Power BI.	—	Documentar y excluir en gitleaks.

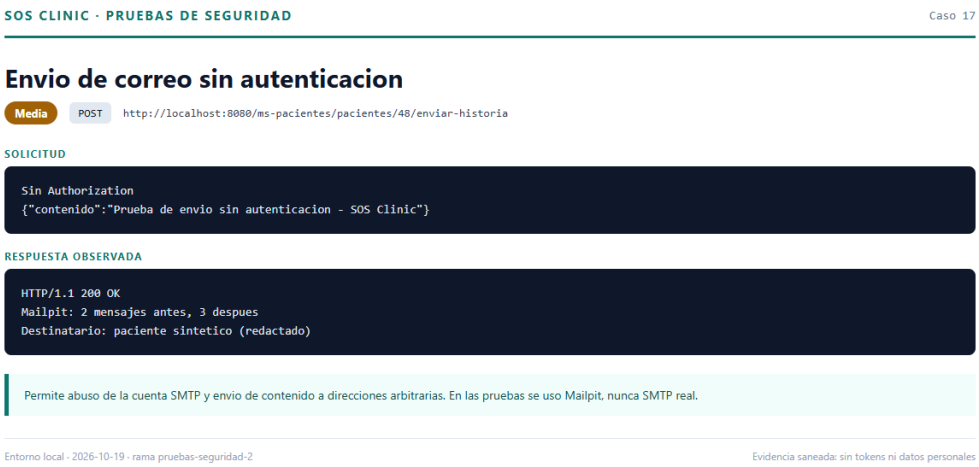


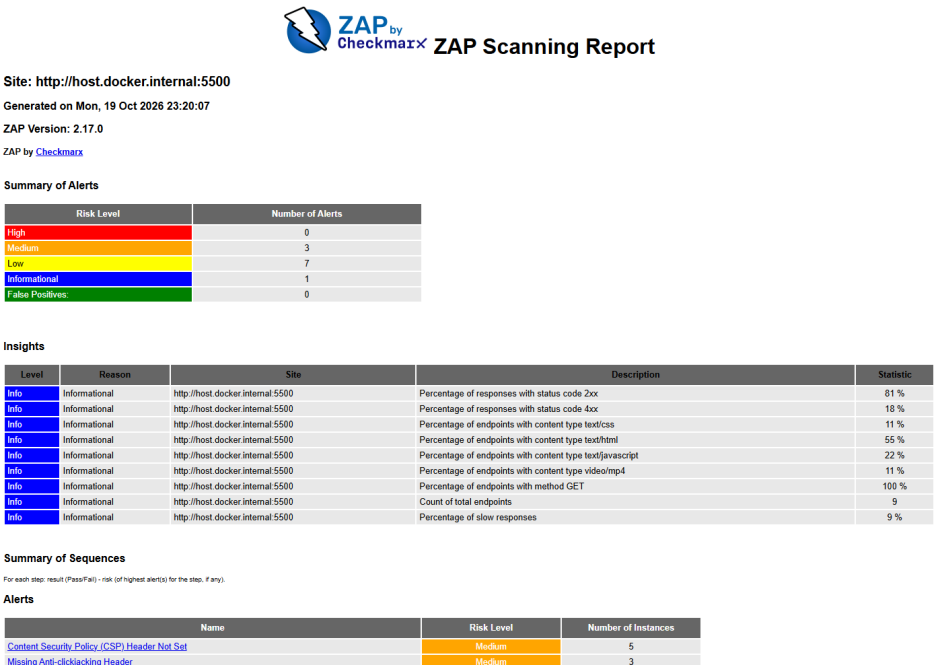
Figura 15: Envío de historia clínica sin autenticación; Mailpit pasó de 2 a 3 mensajes.



Figura 16: Enumeración de usuarios por mensajes diferentes en el login.

5.4 Causa raíz

- **Seguridad descentralizada.** Cada microservicio decide si protege sus rutas; pacientes y triage simplemente no lo hacen, y el gateway no impone una política común.
- **Valores por defecto inseguros.** El secreto JWT, las credenciales de desarrollo y `show-sql` están pensados para comodidad local, pero se versionan y se mezclan con el código de la aplicación.
- **Confianza en el cliente.** La navegación y la sesión se controlan en JavaScript, y el renderizado usa `innerHTML`.
- **Ausencia de puertas automáticas.** Semgrep no encontró los fallos principales y gitleaks no detectó la contraseña del README; no hay gates que bloqueen secretos, SCA crítico ni pruebas de autorización.
- **Agente tratado como consumidor interno confiable.** El agente consulta datos clínicos y servicios externos sin identidad, límites ni autorización por herramienta.



ZAP by Checkmarx ZAP Scanning Report

Site: http://host.docker.internal:5500

Generated on Mon, 19 Oct 2026 23:20:07

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	3
Low	7
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://host.docker.internal:5500	Percentage of responses with status code 2xx	81 %
Info	Informational	http://host.docker.internal:5500	Percentage of responses with status code 4xx	18 %
Info	Informational	http://host.docker.internal:5500	Percentage of endpoints with content type text/css	11 %
Info	Informational	http://host.docker.internal:5500	Percentage of endpoints with content type text/html	55 %
Info	Informational	http://host.docker.internal:5500	Percentage of endpoints with content type text/javascript	22 %
Info	Informational	http://host.docker.internal:5500	Percentage of endpoints with content type video/mp4	11 %
Info	Informational	http://host.docker.internal:5500	Percentage of endpoints with method GET	100 %
Info	Informational	http://host.docker.internal:5500	Count of total endpoints	9
Info	Informational	http://host.docker.internal:5500	Percentage of slow responses	9 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	5
Missing Anti-clickjacking Header	Medium	3

Figura 17: Resumen del reporte ZAP baseline: 0 altas, 3 medias, 7 bajas y 1 informativa.

6. Remediación priorizada

La tabla 15 ordena las acciones por impacto y esfuerzo. Las cuatro primeras concentraban la mayor reducción de riesgo y son, en su mayoría, cambios de configuración o de filtros compartidos. La mayoría de estas acciones se aplicó después de la evaluación; el estado exacto y las pruebas de verificación se presentan en la sección 7.

Tabla 15: Plan de remediación priorizado.

Prioridad	Acción	Hallazgos	Esfuerzo
1	Autenticar y autorizar pacientes y triage; validar JWT en el gateway.	H-02, H-03, H-09, H-23	Medio
2	Restringir `/admin/**` a administradores y separar la gestión de usuarios.	H-04	Bajo
3	Eliminar el secreto JWT por defecto y rotarlo.	H-01	Bajo
4	Mover `env` fuera del repositorio y rotar Gmail y Groq.	H-05, H-13	Bajo
5	Actualizar Tomcat, FreeMarker, Jackson y HttpComponents.	H-06, H-14	Medio
6	Rate limiting y bloqueo en login y 2FA.	H-12, H-15	Medio
7	Corregir XSS y mass assignment.	H-07, H-08	Medio
8	Autenticar y limitar el agente IA.	H-10, H-21	Medio
9	Restringir CORS y añadir cabeceras de seguridad.	H-11, H-18	Bajo
10	Endurecer infraestructura, logs y manejo de errores.	H-19, H-20, H-22	Medio

6.1 Horizonte de ejecución

Corto plazo (0 a 2 semanas).
Aplicar las prioridades 1 a 4 y la 9. Son cambios acotados que eliminan la posibilidad de leer o escribir datos clínicos sin credenciales, forjar tokens administrativos y exponer secretos.

Mediano plazo (2 a 6 semanas).
Actualizar dependencias, implantar rate limiting, corregir XSS y mass assignment, autenticar el agente y endurecer la infraestructura. Añadir pruebas de regresión para cada hallazgo.

Largo plazo (6 semanas o más).
Incorporar threat modeling al ciclo de desarrollo, adoptar ASVS como lista de verificación, establecer una política de dependencias y ejecutar ZAP/SCA/secret scanning de forma continua.

6.2 Verificación

Cada corrección debe verificarse con la misma evidencia que permitió detectar el fallo. La tabla 16 propone las pruebas de aceptación.

Tabla 16: Pruebas de aceptación sugeridas para las correcciones principales.

Corrección	Prueba de aceptación	Resultado esperado
Autenticación de APIs	Repetir `smoke-sin-token` y los recorridos IDOR.	401 en todos los endpoints clínicos.
Autorización administrativa	Token de `doctor_triage` contra `/admin/**`.	403; token admin: 200.
Secreto JWT	Intentar firmar con el valor por defecto y con firma alterada.	401 en ambos casos.
Secretos	Ejecutar gitleaks en árbol e historial.	0 hallazgos reales.
Dependencias	Ejecutar Trivy SCA y comparar contra el umbral.	0 críticas y 0 altas sin excepción.
2FA	20 intentos fallidos y luego el código correcto.	Bloqueo temporal; no aceptación.
XSS	Repetir el payload en pacientes y seguimiento con Playwright.	Sin diálogo; salida codificada.
Mass assignment	Enviar `idPersonal` ajeno e inexistente.	Campo ignorado; 400/404.
CORS	Origen `evil.local` y origen permitido.	Sin cabecera para el origen no permitido.
Agente IA	Llamar sin token y con token de rol limitado.	401/403; herramientas acotadas.

6.3 Controles preventivos

- **CI/CD.** Ejecutar gitleaks, Semgrep, Trivy y ZAP baseline en cada *merge request*; bloquear críticas y altas sin justificación.
- **Pruebas de seguridad.** Añadir pruebas MockMvc para 401/403, validación de JWT, mass assignment y codificación de salida.
- **Secretos.** Usar variables de entorno o un gestor de secretos; prohibir valores por defecto en `application.properties`.
- **Dependencias.** Fijar versiones, revisar CVEs y mantener un *software bill of materials* (SBOM).
- **Observabilidad.** Auditar accesos a historias clínicas, cambios de rol y creación de usuarios administrativos; no registrar tokens ni códigos.
- **Diseño.** Aplicar denegación por defecto, mínimo privilegio y autorización a nivel de objeto en todos los servicios.

7. Estado de remediación

Después de la evaluación base en la rama `pruebas-seguridad-2`, se aplicaron correcciones en `main` y se documentaron en `seguridad/REMEDIACION-2026-10-19.md` [2]. La rama de pruebas conserva la evidencia del estado anterior; la rama principal contiene el código corregido y las pruebas de verificación. En esta sección se resume el estado de cada hallazgo, las pruebas ejecutadas, la migración necesaria y los riesgos residuales.

7.1 Resumen de estado

Tabla 17: Resultado de la remediación por estado.

Estado	Cantidad	Hallazgos
Remediado	22	H-01 a H-04, H-06 a H-17, H-19 a H-24.
Mitigado	1	H-05; requiere rotar credenciales externas.
Parcial	1	H-18; CSP todavía con `unsafe-inline`.
Documentado	1	H-25; falso positivo con allowlist.
Total	**25**	

7.2 Estado por hallazgo

Tabla 18: Estado de remediación de los 25 hallazgos.

ID	Hallazgo	Estado	Evidencia / prueba
H-01	Secreto JWT por defecto	**Remediado**	Fallback eliminado; `APP_JWT_SECRET` obligatorio; token viejo 401, token nuevo 200.
H-02	Lectura sin autenticación	**Remediado**	Sin token: 401 en gateway y en puertos directos.
H-03	Escritura y altas sin autenticación	**Remediado**	POST/PUT sin token: 401; roles clínicos requeridos.
H-04	Escalada en `/admin/**`	**Remediado**	`doctor_triage`: 403; `administrador`: 200.
H-05	Archivo `env` con credenciales	**Mitigado**	`env/.env` ignorados; `.env` local no versionado; rotar Gmail y Groq.
H-06	Dependencias críticas	**Remediado**	Tomcat 11.0.25, FreeMarker 2.3.35; Trivy sin críticas ni altas.
H-07	XSS almacenado	**Remediado**	Playwright: 0 diálogos y 0 `<img>` inyectadas.
H-08	Mass assignment	**Remediado**	`SignosVitalesRequest`; `idPersonal` del JWT; fecha y estado del servidor.
H-09	IDOR	**Remediado (por rol)**	Sin token: 401; roles clínicos/lectura según matriz; sin propiedad por episodio.
H-10	Agente IA sin autenticación	**Remediado**	`/agente/**` exige JWT; contexto del cliente ignorado; sin nombres ni cédulas a Groq.
H-11	CORS permisivo	**Remediado**	`evil.local`: 403; `localhost:5500`: 200 con ACAO.
H-12	2FA sin límite	**Remediado**	BCrypt, `SecureRandom`, invalidación previa; 5 fallos: 429.
H-13	Contraseña en README	**Remediado (árbol)**	Valor reemplazado por placeholder; el historial conserva el valor antiguo, rotar.

ID	Hallazgo	Estado	Evidencia / prueba
H-14	Dependencias altas	**Remediado**	Jackson 2.21.7/3.1.7, HttpCore 5.4.3, HttpClient 4.5.13, HttpClient5 5.6.3; Trivy sin altas.
H-15	Login sin bloqueo	**Remediado**	5 fallos: 429; el login correcto queda bloqueado hasta cumplir la ventana.
H-16	Correo sin autenticación	**Remediado**	Endpoint exige JWT y rol clínico.
H-17	Enumeración de usuarios	**Remediado**	Mensajes genéricos y BCrypt dummy para usuarios inexistentes o inactivos.
H-18	Cabeceras ausentes	**Parcial**	Cabeceras en gateway y frontend; CSP con `unsafe-inline` por los scripts existentes.
H-19	Infraestructura expuesta	**Remediado**	Puertos en 127.0.0.1; credenciales por `.env`; sin root/root en scripts.
H-20	`show-sql` y logs	**Remediado**	`show-sql=false`; sin códigos 2FA ni password generada en logs.
H-21	Voz del agente sin autenticación	**Remediado**	`/agente/transcribir` exige JWT; límite de 10 MB y rate limit.
H-22	Errores 500	**Remediado**	Manejadores globales; JSON inválido 400; conflictos 409.
H-23	Navegación solo en cliente	**Remediado**	Autorización real en gateway y servicios; el frontend queda como UX.
H-24	Rol de analítica	**Remediado**	`coordinador_turnos` se normaliza a `ROLE_COORDINACION`.
H-25	Falso positivo Power BI	**Documentado**	Allowlist del enlace público en `.gitleaks.toml`.

7.3 Pruebas de verificación

Tabla 19: Pruebas ejecutadas después de la remediación.

Prueba	Resultado
`mvn test` en los siete módulos	Todos en verde.
Trivy SCA	0 críticas y 0 altas en los siete `pom.xml`.
gitleaks sobre el árbol	Sin hallazgos; `.env` local excluido y no versionado.
E2E sin token	401 en pacientes, triage, admin y agente.
E2E por roles	Admin 200 en `/admin/personal`; doctor 403; coordinación lectura 200 y escritura 403; PDF 200.
2FA	Código correcto 200; cinco fallos 429; código correcto bloqueado 429.
Login	Cinco fallos 429; mensajes genéricos.
CORS	Origen permitido 200; origen no permitido 403.
Mass assignment	`idPersonal` del cuerpo ignorado; se guarda el del JWT.
XSS	Playwright sin diálogos en pacientes, seguimiento y lista de espera.
Agente	Respuesta sin nombres de pacientes; rate limit 20/min.

7.4 Migración y riesgos residuales

La migración SQL de `infra/migraciones/` amplía la columna del código de verificación a `VARCHAR(255)` para almacenar el hash BCrypt, y crea o actualiza las columnas de turno y de 2FA si faltan. Debe ejecutarse sobre las bases existentes antes de desplegar la versión corregida.

- El logout sigue siendo local: el JWT es sin estado y no hay lista de revocación.

- El límite de intentos está en memoria; con varias instancias debe moverse a Redis o a columnas de base de datos.
- H-09 quedó cerrado por rol, no por propiedad de cada episodio.
- La CSP conserva `unsafe-inline`; una CSP estricta requiere retirar los scripts y `onclick` inline del frontend.
- El entorno local usa HTTP sin HSTS; en producción debe ir detrás de TLS.
- Los secretos antiguos siguen en el historial de Git. Deben rotarse; borrarlos del árbol no los invalida.
- El agente envía a Groq solo conteos y agregados; no nombres ni cédulas.

Alcance de la verificación. Las pruebas de esta sección provienen del documento de remediación y de la ejecución local sobre `main`. No sustituyen una auditoría independiente ni una re-evaluación completa de todos los casos de la matriz original.

8. Conclusiones

La evaluación base confirmó que SOS Clinic priorizaba la funcionalidad sobre la seguridad: las APIs clínicas no autenticaban, las rutas administrativas no autorizaban por rol y el secreto JWT estaba versionado. El riesgo global medido en la rama `pruebas-seguridad-2` era **crítico**. Después de las correcciones aplicadas en `main`, **22 hallazgos quedaron remediados, 1 mitigado (H-05), 1 parcial (H-18) y 1 documentado (H-25)**. El riesgo residual baja de forma significativa, pero no desaparece.

Los objetivos de la evaluación se cumplieron: se identificaron secretos y configuraciones inseguras, se verificó la ausencia de autenticación y autorización en los servicios de negocio, se demostró el XSS almacenado y el abuso del agente IA, y se consolidaron 25 temas únicos con evidencia reproducible. La matriz de hallazgos, las capturas y los resultados crudos permiten repetir las pruebas; la sección 7 añade el estado de las correcciones y las pruebas de verificación ejecutadas en `main`.

También quedan limitaciones claras. El entorno es local y de desarrollo; ZAP se ejecutó en modo pasivo; el SCA dependió del caché local de Maven; Trivy no evaluó `docker-compose.yml`; y la verificación posterior se apoya en `mvn test`, Trivy, gitleaks y pruebas E2E, no en una auditoría independiente. Por tanto, los resultados no deben interpretarse como una certificación de producción ni como una garantía de ausencia de vulnerabilidades.

La recomendación final es mantener la seguridad como una propiedad transversal y cerrar los riesgos residuales: revocación de sesiones o logout con estado, rate limiting distribuido, autorización por propiedad de cada episodio, CSP estricta sin `unsafe-inline`, TLS con HSTS en producción y rotación de los secretos que permanecen en el historial de Git. Con esas acciones, y una re-evaluación independiente, el prototipo puede pasar de un riesgo crítico a un riesgo controlado.

Referencias

[1] S. Chacón Mantilla, O. Castellanos López y S. Ramírez Cubillos. *SOS-Clinic, rama* `pruebas-seguridad-2`. GitLab, 2026. https://gitlab.com/SebasChakon/sos-clinic/-/tree/pruebas-seguridad-2

[2] S. Chacón Mantilla, O. Castellanos López y S. Ramírez Cubillos. *Remediación de hallazgos de seguridad — 2026-10-19*. Rama `main`, 2026. `seguridad/REMEDIACION-2026-10-19.md`

[3] OWASP Foundation. *Web Security Testing Guide v4.2*. 2020. https://owasp.org/www-project-web-security-testing-guide/

[4] OWASP Foundation. *OWASP API Security Top 10 2023*. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[5] OWASP Foundation. *Application Security Verification Standard 4.0.3*. 2021. https://owasp.org/www-project-application-security-verification-standard/

[6] ISO/IEC. *ISO/IEC 27002:2022. Information security, cybersecurity and privacy protection — Information security controls*. 2022.

[7] FIRST. *Common Vulnerability Scoring System v4.0: Specification Document*. 2023. https://www.first.org/cvss/v4-0/

[8] PTES. *Penetration Testing Execution Standard*. http://www.pentest-standard.org/

[9] MITRE. *Common Weakness Enumeration*. https://cwe.mitre.org/

[10] NIST. *SP 800-115. Technical Guide to Information Security Testing and Assessment*. 2008. https://csrc.nist.gov/pubs/sp/800/115/final

[11] OWASP Foundation. *OWASP ZAP*. https://www.zaproxy.org/

[12] Aqua Security. *Trivy*. https://trivy.dev/

[13] Semgrep. *Semgrep*. https://semgrep.dev/

[14] Gitleaks. *Gitleaks*. https://github.com/gitleaks/gitleaks

[15] OWASP Foundation. *OWASP Cheat Sheet Series*. https://cheatsheetseries.owasp.org/

[16] Spring. *Spring Security Reference*. https://docs.spring.io/spring-security/reference/

A. Detalle del análisis de dependencias

Trivy reportó 148 ocurrencias en los siete módulos. Al deduplicar por CVE, paquete y versión quedan 22 vulnerabilidades únicas: 4 críticas, 12 altas y 6 medias. La tabla 20 agrupa las CVEs por paquete y versión instalada.

Tabla 20: Detalle agrupado del SCA.

Paquete	Versión	CVEs	Severidad	Corrección
tomcat-embed-core	11.0.24	65182, 65905, 68525	3 críticas	11.0.25
freemarker	2.3.34	84939	1 crítica	2.3.35
jackson-databind	2.21.5	68497, 91776, 91777, 19032, 83557	3 altas, 2 medias	2.21.6 / 2.22.2
jackson-databind	3.1.5	68497, 91776, 91777, 19032, 83557	3 altas, 2 medias	3.1.6 / 3.2.2
jackson-core	2.21.5	89407, 89425	2 altas	2.21.7 / 2.22.3
jackson-core	3.1.5	89407, 89425	2 altas	3.1.7 / 3.2.3
httpcore5	5.3.6	54399	1 alta	5.4.3
httpcore5-h2	5.3.6	54428	1 alta	5.4.3
httpclient	4.5.3	2020-13956	1 media	4.5.13
httpclient5	5.5.2	64607	1 media	5.6.3

Todos los CVEs pertenecen al prefijo CVE-2026 salvo CVE-2020-13956. La columna de corrección indica la primera versión que Trivy marcó como no vulnerable; conviene validar la compatibilidad antes de actualizar.

B. Índice de evidencias y resultados crudos

Tabla 21: Índice de las 16 capturas de evidencia.

Archivo	Hallazgo	Contenido
`07_jwt_forjado_admin.png`	H-01	JWT forjado y 200 en `/admin/personal`.
`08_escalada_admin.png`	H-04	Cuenta administradora creada por `doctor_triage`.
`09_2fa_sin_limite.png`	H-12	Diez intentos fallidos y aceptación posterior.
`10_cors_permisivo.png`	H-11	Origen `evil.local` reflejado con credenciales.
`11_pacientes_sin_token.png`	H-02	Lectura clínica sin token.
`12_escritura_sin_token.png`	H-03	Escritura y alta sin token.
`13_mass_assignment.png`	H-08	`idPersonal` aceptado desde el cuerpo.
`14_idor.png`	H-09	Recorrido de IDs ajenos.
`15_xss_pacientes.png`	H-07	Payload XSS en la tabla de pacientes.
`15b_xss_seguimiento.png`	H-07	Diálogo `alert("1")` en seguimiento.
`16_agente_sin_token.png`	H-10	Agente con datos clínicos sin token.
`17_correo_sin_token.png`	H-16	Envío de historia sin autenticación.
`18_enumeracion_usuarios.png`	H-17	Mensajes distintos para usuario inactivo.
`19_sca_trivy.png`	H-06, H-14	Resumen del SCA.
`20_secretos_gitleaks.png`	H-05, H-13	Secretos detectados.
`21_zap_baseline.png`	H-18	Resumen del ZAP baseline.

Los reportes crudos (JSON, CSV y HTML) están en `seguridad/resultados/`: gitleaks, Semgrep, Trivy SCA, ZAP, smoke tests, recorridos IDOR, diálogos XSS y la matriz `hallazgos-consolidados.csv`.

Los comandos exactos de gitleaks, Semgrep, Trivy y ZAP, junto con los scripts de smoke tests, IDOR y el generador de JWT, están en `seguridad/utilidades/` y `seguridad/resultados/` de la rama. Los tokens, contraseñas y cuerpos con datos personales se omitieron de las capturas y de los reportes versionados.